

Appendix C: AI-Generated Code Security Checklist

Use before merging any PR with AI-generated auth, authorization, input handling, or external service integration. Approximately 10 minutes per PR.

INPUT VALIDATION

- ☐ All inputs from external sources validated for type, format, and range
- ☐ String inputs with expected formats (UUID, email, URL) validated before use
- ☐ Numeric inputs have minimum and maximum bounds enforced
- ☐ Array/list inputs have maximum length limits
- ☐ File uploads validate MIME type, extension, AND file content

AUTHENTICATION

- ☐ Every protected endpoint checks authentication before processing
- ☐ Session tokens validated on every request — not just on login
- ☐ Password reset flows use time-limited, single-use tokens
- ☐ Login and reset endpoints have rate limiting applied
- ☐ Auth failure messages unified — no distinction between 'user not found' and 'wrong password' (prevents email enumeration)

AUTHORIZATION

- ☐ Every resource endpoint checks ownership/permission before returning data
- ☐ Database queries for user-owned resources include user ID as a filter
- ☐ Admin endpoints protected with role checks, not just authentication
- ☐ Suspended or deleted account states checked in access control logic

DATA HANDLING

- ☐ Sensitive data (passwords, tokens) never logged
- ☐ Error messages to clients do not include stack traces
- ☐ Database error strings caught server-side — not forwarded to clients
- ☐ Secrets read from environment variables — not hardcoded

EXTERNAL SERVICES

- ☐ Webhook payloads signature-verified before processing
- ☐ External API calls have explicit timeout values
- ☐ External API errors handled — do not crash the application
- ☐ Library versions pinned and checked against known CVEs

CRYPTOGRAPHY

- ☐ Passwords hashed with bcrypt/Argon2/scrypt — not MD5 or SHA-256 alone
- ☐ Random tokens use cryptographically secure generation
- ☐ TLS enforced for all external communications

Index

Key terms with chapter references (Ch. = chapter number). For exact page numbers, use the Table of Contents or Ctrl+F in Word.

A

- Agentic loop — Ch. 8
- Agentic mode configuration — Ch. 8
- Alembic migrations — Ch. 11
- Async/await migration from callbacks — Ch. 7
- Audit → Protect → Transform framework — Ch. 7
- Auth.js v5 — Ch. 13

C

- Callback migration to async — Ch. 7
- Characterization tests — Ch. 7
- Checkpoint pattern (human-in-the-loop) — Ch. 8
- CI/CD pipeline generation — Ch. 10
- CLAUDE.md (individual) — Ch. 3
- CLAUDE.md (team template) — Ch. 14, App B
- Claude Code, installing — Ch. 1
- Code generation mastery — Ch. 4
- Code review, AI-assisted — Ch. 14
- Code smells — Ch. 7
- Context cliff (AI failure mode) — Ch. 15
- Copyright of AI-generated code — Ch. 15
- cron scheduling — Ch. 12

D

- Database MCP (PostgreSQL) — Ch. 9
- Debugging protocol — Ch. 5
- Docker, multi-stage builds — Ch. 10
- Drizzle ORM — Ch. 13

E

- Error handling conventions — Ch. 4, 14
- Error message leakage — Ch. 15
- Email automation project — Ch. 12

F

- Fan-out orchestration pattern — Ch. 8
- FastAPI backend — Ch. 11
- Filesystem MCP — Ch. 9
- Freelance workflow (scope/quote/build) — Ch. 13

G

- GitHub Actions CI workflow — Ch. 10
- GitHub Actions templates — Ch. 10
- GitHub MCP — Ch. 9
- Guard clauses (early return pattern) — Ch. 7

H

- Hooks (Claude Code lifecycle) — Ch. 8
- Human-in-the-loop checkpoint — Ch. 8

I

IDOR vulnerability — *Ch. 15*
Input validation, missing — *Ch. 15*
Integration tests — *Ch. 6*

L

Legacy code, refactoring — *Ch. 7*
Licensing of AI output — *Ch. 15*

M

MCP (Model Context Protocol) — *Ch. 9, 12*
mcp.json configuration — *Ch. 9, 12*
Memory files — *Ch. 3*
Migrations (database) — *Ch. 11, 13*
Multi-stage Docker builds — *Ch. 10*

N

Next.js App Router — *Ch. 13*
Notion MCP — *Ch. 9, 12*

O

Onboarding new developers — *Ch. 14*
OWASP Top 10 — *Ch. 15*

P

Parallel subagents (fan-out) — *Ch. 8*
Parity tests (migration) — *Ch. 7*
Payment integration (Stripe) — *Ch. 11*
Permission helpers — *Ch. 13*
PR review workflow — *Ch. 14*
Prompt structure (five components) — *Ch. 2*

R

Race conditions (AI failure mode) — *Ch. 15*
Railway deployment — *Ch. 11*
Rate limiting, missing — *Ch. 15*
React landing page — *Ch. 11*
Refactoring monolith project — *Ch. 7*
Reviewer pattern (subagents) — *Ch. 8*

S

Safe deploy protocol — *Ch. 10*
SaaS landing page project — *Ch. 11*
Security checklist — *Ch. 15, App C*
Security risks of AI code — *Ch. 15*
Senior engineer review lens — *Ch. 15*
Server Actions (Next.js) — *Ch. 13*
Slash commands — *Ch. 8, 14*
Slack MCP — *Ch. 9, 12*
Spaghetti functions — *Ch. 7*
Strangler fig pattern — *Ch. 7*
Stripe Checkout integration — *Ch. 11*
Subagent orchestration — *Ch. 8*

T

Team adoption stages — *Ch. 14*

Team CLAUDE.md template — *Ch. 14, App B*
Terraform configuration generation — *Ch. 10*
Test-first prompting — *Ch. 6*
Testing requirements — *Ch. 6*
Trust boundary map — *Ch. 15*
TypeScript migration — *Ch. 7*

V

Vercel deployment — *Ch. 13*

W

Web search MCP — *Ch. 9*
Webhook handler (Stripe) — *Ch. 11*
Workflow automation project — *Ch. 12*